Rushabh Jain
D15B-23

MPL-2 (Assignment)

A PWA is a web application that combines both web & mobile apps to deliver a seamless. PWA experience

They are designed to be fast, reliable & engaging across various devices

-Significance -

- Platform Independence
- Improved Performance
- Offline functionality
- No app store dependencies
- Engaging User Experience

- Difference from traditional mobile apps (TMA)

- PWA is installed via browser while TMA are installed via playstore
- PWA takes up less storage
- PWA are fast & browser dependent, TMA are optimised for hardwares
- PWA has limited offline access.

Responsiveness is an approach to ensures that web pages adapt to different screen sizes & orientations using flexible grids.

Importance -

- Ensures a consistent user experience across differe-

-nt devices
. Eliminates need for multiple codebases for d
-esent devices
. Enhances usability by making content reach
able on screen

Responsive	Fluid	Adaptive
Uses CSS media qrs to adjst layout	Uses % for elemnts for scale	Uses predefined layou for diff screen siz
Highly flexible	Completely Flexible	Fixed at breakpoints
Efficient performance	Smooth Scaling	May cause layout shifts

Q3

→ Life cycle phases -

1) Registration
. A service worker is registered using JS in web ap
The browser checks if SW exists ; if new or updated, it moves to the next phase

```
if ('service worker' in navigator){
    navigator.serviceWorker.register('/sw.js')
then(() => console.log('registered')
}
```

2) Installation
Occurs when the service worker is first downloaded

- Executes install event & caches assets in this phase using cache.addAll()

```
self.addEventListener('install', event => {
    event.waitUntil(
        caches.open('v') then cache => {
    return cache.addAll
});
```

Activation

The 'activate' event installs after installation & ensures old caches cleared if necessary.
Takes control of page using self.clients.claim().

```
self.addEventListener('activate', event => {
    event.waitUntil(
    caches.keys(), then (keys => {
    return Promise.all [keys.filter (key => key !== 'v')]
});
}
```

3) Fetching & Updates

listens to fetch event to intercept network requests and updates the new service workes if found.

```
self.addEventListener('fetch', event => {
    event.respondWith(
        caches.match(event.request)
    );
});
```

Q4

→ · Indexed DB is a low level, asynchronous database API that allows service workers to store & manage structured data efficiently
· It is useful for handling large amounts of offline data, like user preferences, cached API responses, or application test.

- Uses of Indexed DB -
· Persistent storage - Data remains even after browser is closed
· Supports Complex Data - Can store objects, arrays & files
· Efficient Fetching - Faster access than Cache API
· Offline API caching - Store & retrieve API response for offline access
· Large Asset Storage - Caches images, videos & documents efficiently.